2026-09-29 11:05:25 You

告诉我算法竞赛中stl常用的一些东西

ChatGPT gpt-5-6-thinking

算法竞赛里，STL 真正高频的其实就那么一批。你先把这些熟练，已经能覆盖大部分题。

1. `vector`：动态数组

最常用的容器之一。

```cpp
vector<int> a;

a.push_back(3);      // 尾部加入
a.pop_back();        // 删除最后一个
a.size();            // 大小
a.empty();           // 是否为空
a.clear();           // 清空

a[0];                // 下标访问
a.front();           // 第一个
a.back();            // 最后一个
```

初始化：

```cpp
vector<int> a(n);        // n个0
vector<int> a(n,5);      // n个5
vector<vector<int>> g(n);// 二维vector, 图常用
```

遍历：

```cpp
for(int x:a)
    cout << x << ' ';
```

或者：

```cpp
```

```cpp
for(auto &x:a)
    cin >> x;
```

`&` 很重要，表示直接修改原数组。

---

2. `pair`：存两个值

```cpp
pair<int,int> p={3,5};

cout << p.first;
cout << p.second;
```

更常写：

```cpp
pair<int,int> p={x,y};
```

比如图：

```cpp
vector<pair<int,int>> g[N];
```

表示：

```text
{终点, 权值}
```

也可以：

```cpp
auto [x,y]=p;
```

这个很好用：

```cpp
pair<int,int> p={3,5};

auto [x,y]=p;
```

```cpp
cout << x << ' ' << y;
```

---

3. string

```cpp
string s;

cin >> s;

s.size();
s.empty();
s.push_back('a');
s.pop_back();

s.substr(l,len);
```

比如：

```cpp
string s="abcdef";

cout << s.substr(2,3);
```

输出：

```text
cde
```

查找：

```cpp
s.find("abc");
```

如果没找到：

```cpp
s.find("abc")==string::npos
```

---

4. set：自动排序 + 去重

你刚才的题就用了。

```cpp
set<int> s;

s.insert(5);
s.insert(2);
s.insert(5);
```

最后实际上是：

```text
2 5
```

常用：

```cpp
s.insert(x);
s.erase(x);
s.count(x);
s.find(x);
s.size();
```

最大最小：

```cpp
*s.begin()          // 最小
*prev(s.end())      // 最大
*s.rbegin()         // 最大
```

查找：

```cpp
auto it=s.lower_bound(x);
```

表示：

> 第一个 >= x 的位置

```cpp
auto it=s.upper_bound(x);
```

表示：

> 第一个 > x 的位置

复杂度基本都是：

```text
O(log n)
```

---

5. `multiset`：可以有重复元素的 set

```cpp
multiset<int> s;

s.insert(3);
s.insert(3);
s.insert(3);
```

里面真的有三个 `3`。

这个地方特别重要：

```cpp
s.erase(3);
```

会把 **所有 3 都删掉**。

如果只想删一个：

```cpp
auto it=s.find(3);

if(it!=s.end())
    s.erase(it);
```

你刚才 Rectangle Cutting 那题就是：

```cpp
cc.erase(cc.find(R-L));
```

只删除一个长度。

---

6. `map`：键值对

可以理解成：

```text
key -> value
```

比如记录每个数字出现次数：

```cpp
map<int,int> mp;

mp[x]++;
```

例如：

```cpp
mp[5]=3;
```

就是：

```text
5 -> 3
```

遍历：

```cpp
for(auto [x,y]:mp)
{
    cout << x << ' ' << y << '\n';
}
```

`map` 会按照 key 排序。

复杂度：

```text
O(log n)
```

---

7. `unordered_map`

和 `map` 类似：

```cpp
```

```cpp
unordered_map<int,int> mp;

mp[x]++;
```

区别是：

```text
map
有序
O(log n)

unordered_map
无序
平均 O(1)
```

不过竞赛里 `unordered_map` 有时候会被特殊数据卡掉，所以不是所有时候都推荐它。

初学阶段：

```text
不确定就 map
```

---

8. `queue`：队列

先进先出。

```cpp
queue<int> q;

q.push(x);
q.front();
q.pop();
q.empty();
q.size();
```

BFS 高频：

```cpp
queue<int> q;

q.push(s);

while(!q.empty())
```

```
{
    int u=q.front();
    q.pop();

    // ...
}
```

---

9. `stack`：栈

后进先出。

```cpp
stack<int> s;

s.push(x);
s.top();
s.pop();
s.empty();
```

常见于：

```text
括号匹配
单调栈
表达式
DFS模拟
```

---

10. `priority_queue`：优先队列 / 堆

默认大根堆：

```cpp
priority_queue<int> q;

q.push(3);
q.push(8);
q.push(5);

cout << q.top();
```

输出：

```text
```

```
8
```

小根堆：

```cpp
priority_queue<int,vector<int>,greater<int>> q;
```

Dijkstra 特别常见：

```cpp
priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>
> q;
```

---

11. `deque`：双端队列

两边都能插入删除：

```cpp
deque<int> q;

q.push_back(x);
q.push_front(x);

q.pop_back();
q.pop_front();

q.front();
q.back();
```

常见：

```text
单调队列
滑动窗口
01 BFS
```

---

12. `sort`

超级高频。

```cpp
sort(a.begin(),a.end());
```

从小到大。

从大到小：

```cpp
sort(a.begin(),a.end(),greater<int>());
```

数组：

```cpp
sort(a+1,a+n+1);
```

排序 `pair`：

```cpp
vector<pair<int,int>> a;

sort(a.begin(),a.end());
```

默认：

先比较 `first`，相同再比较 `second`。

自定义：

```cpp
sort(a.begin(),a.end(),[](auto a,auto b)
{
    return a.second<b.second;
});
```

---

13. `lower_bound`

二分题神器。

前提：

> 数组已经排序。

```cpp
```

```cpp
auto it=lower_bound(a.begin(),a.end(),x);
```

返回：

```text
第一个 >= x
```

如果想得到下标：

```cpp
int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

数组版本：

```cpp
int pos=lower_bound(a+1,a+n+1,x)-a;
```

---

14. upper_bound

```cpp
upper_bound(a.begin(),a.end(),x);
```

返回：

```text
第一个 > x
```

所以：

```cpp
lower_bound
```

是：

```text
>= x
```

而：

```cpp
upper_bound
```

是：

```text
> x
```

这个一定要记住。

---

15. binary_search

判断有没有：

```cpp
if(binary_search(a.begin(),a.end(),x))
{
    cout << "YES";
}
```

前提也是：

```text
数组有序
```

---

16. reverse

翻转：

```cpp
reverse(a.begin(),a.end());
```

字符串也可以：

```cpp
reverse(s.begin(),s.end());
```

---

17. max_element / min_element

找最大最小：

```cpp
int mx=*max_element(a.begin(),a.end());
int mn=*min_element(a.begin(),a.end());
```

找位置：

```cpp
int pos=max_element(a.begin(),a.end())-a.begin();
```

---

18. accumulate

求和：

```cpp
#include <numeric>

ll sum=accumulate(a.begin(),a.end(),0LL);
```

注意最好写：

```cpp
0LL
```

而不是：

```cpp
0
```

否则如果数组和很大，可能按 `int` 算。

---

19. unique

经典去重：

```cpp
sort(a.begin(),a.end());

a.erase(unique(a.begin(),a.end()),a.end());
```

一定记住：

```cpp
unique()
```

本身不会真的删元素。

一般直接背：

```cpp
sort(a.begin(),a.end());
a.erase(unique(a.begin(),a.end()),a.end());
```

离散化里天天用。

---

20. next_permutation

生成下一个排列：

```cpp
vector<int> a={1,2,3};

do
{
    for(int x:a)
        cout << x;
    cout << '\n';
}
while(next_permutation(a.begin(),a.end()));
```

会得到：

```text
123
132
213
231
312
321
```

适合：

```text
n 很小
暴力枚举排列
```

21. prev / next

你刚才已经接触到了：

```cpp
auto it=s.find(x);

prev(it);  // 前一个
next(it);  // 后一个
```

比如：

```cpp
int L=*prev(it);
int R=*next(it);
```

注意别越界：

```cpp
prev(s.begin())
```

是不允许的。

```cpp
next(prev(s.end()))
```

之类也要注意边界。

---

22. begin / end / rbegin

对于：

```cpp
set<int> s;
```

有：

```cpp
s.begin()          // 第一个
s.end()            // 最后一个元素的后面
s.rbegin()         // 倒着的第一个，也就是最大值
```

所以：

```cpp
*s.begin()
```

最小值。

```cpp
*s.rbegin()
```

最大值。

注意：

```cpp
*s.end()
```

是错的。

因为 `end()` 指向的是：

```text
最后一个元素后面的位置
```

所以最后一个应该：

```cpp
*prev(s.end())
```

---

23. `count`

`vector/string`：

```cpp
count(a.begin(),a.end(),x);
```

统计 `x` 出现次数。

对于：

```cpp

```

```
set
map
```

也可以：

```cpp
s.count(x);
mp.count(x);
```

常拿来判断存不存在：

```cpp
if(mp.count(x))
{
}
```

---

24. fill

快速赋值：

```cpp
fill(a,a+n,0);
```

vector：

```cpp
fill(a.begin(),a.end(),INF);
```

比如 Dijkstra：

```cpp
fill(dis,dis+N,INF);
```

---

25. memset

这个不是 STL，但竞赛非常常见：

```cpp
memset(a,0,sizeof a);
```

全部置 0。

```cpp
memset(a,-1,sizeof a);
```

全部置 -1。

但是不要随便：

```cpp
memset(a,1,sizeof a);
```

你可能以为会变成 1，其实不会。

---

26. bitset

处理很多 0/1：

```cpp
bitset<1000> b;

b[3]=1;

b.set(5);
b.reset(3);

b.count();
```

位运算题、状态题有时非常强。

你目前最该熟练的

如果按重要程度学习，我建议先把下面这一批练到看到就会写：

```text
vector
pair
string

set
multiset
map

queue
```

```
stack
priority_queue

sort
lower_bound
upper_bound
reverse
unique

begin
end
prev
next
```

尤其是这几个复杂度要有感觉：

操作	复杂度
`vector` 下标访问	O(1)
vector push_back	均摊 O(1)
`sort`	O(n log n)
lower_bound	O(log n)
`set` insert/find/erase	O(log n)
map insert/find	O(log n)
`priority_queue` push/pop	O(log n)
priority_queue top	O(1)
`queue` push/pop	O(1)

你现在这个阶段，我觉得 **`set / multiset / map / priority_queue / lower_bound`** 要重点学，因为中档算法题里出现频率很高。